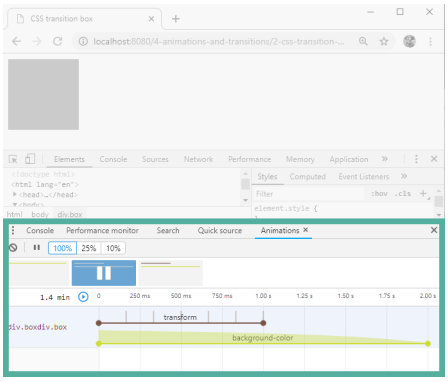
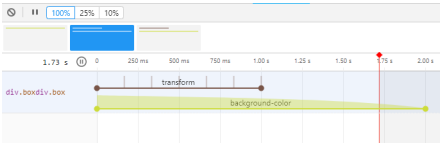
> If you don't see the **Animations** tab, click on the kebab menu on the left and select it from the dropdown options.



animations panel

Once we've triggered our box transition by hovering, we can inspect the animation.



animations panel zoomed

We can see the `transform` transition on top, with six discrete changes, and the `background` animation on the bottom, easing gradually from one color to the next. The `background` transition diagram is twice as wide as the `transform` transition diagram, indicating that it takes twice as long.

This view can be very handy when inspecting, tweaking, and debugging transitions.

Now that we're comfortable with CSS `transition`, let's see how we might use it to animate our charts.

Using CSS transition with a chart

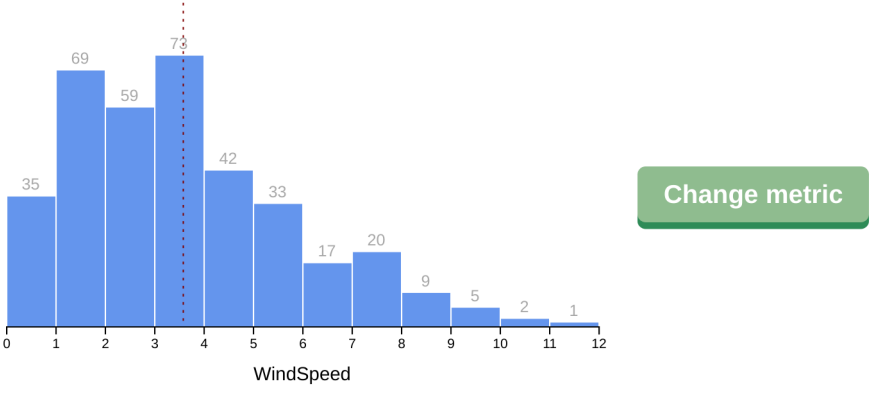
Navigate to the `4-animations-and-transitions/3-draw-bars-with-css-transition` folder. The `index.html` is importing a CSS stylesheet (`styles.css`) and the

`updating-bars.js` file, which is an updated version of our histogram drawing code from **Chapter 3**.

The code should look mostly familiar, but you might notice a few changes. Don't worry about those changes at the moment — they're not important to our main mission.

Let's look inside that `styles.css` file. We can see that we have already set the basic styles for our bars (`.bin rect`), bar labels (`.bin text`), mean line (`.mean`), and x axis label (`.x-axis-label`).

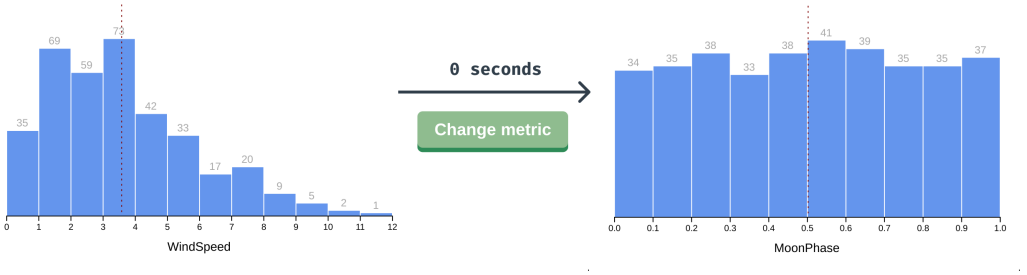
Great! Now that we know the lay of the land, let's point our browser at http://localhost:8080/04-animations-and-transitions/3-draw-bars-with-css-transition/[25] — we should see our histogram and a **Change metric** button.



bars

When we click the button, our chart re-draws with the next metric, but the change is instantaneous.

[25] http://localhost:8080/04-animations-and-transitions/3-draw-bars-with-css-transition/



instant change

Does this next metric have fewer bars than the previous one? Does our mean line move to the left or the right? These questions can be answered more easily if we transition gradually from one view to the other.

Let's add an animation whenever our bars update (`.bin rect`).

```
.bin rect {
    fill: cornflowerblue;
    transition: all 1s ease-out;
}
```

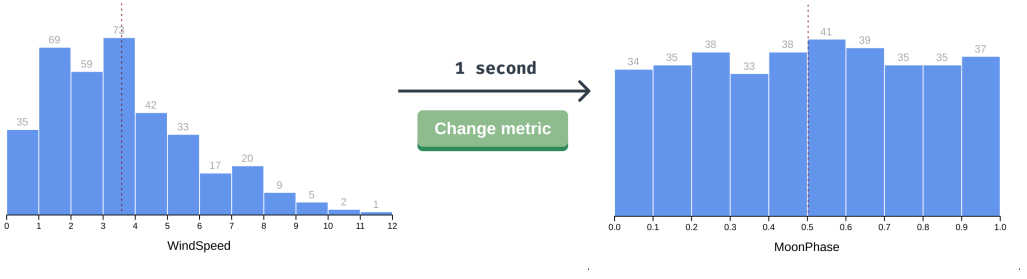
> Note that this CSS transition will currently only work in the Chrome browser. This is because Chrome is the only browser that has implemented the part of the SVG 2 spec[a] which allows `height` as a CSS property, letting us animate the transition.
>
> [a] https://www.w3.org/TR/SVG/styling.html#StylingUsingCSS

Now when we update our metric, our bars shift slowly to one side while changing height — we can see that their `width`, `height`, `x`, and `y` values are animating. This may be fun to watch, but it doesn't really represent our mental model of bar charts. It would make more sense for the bars to change position instantaneously and animate any height differences. Let's only transition the `height` and `y` values.

```
.bin rect {
    fill: cornflowerblue;
    transition: height 1s ease-out,
                      y 1s ease-out;
}
```

> `ease-out` is a good starting point for CSS transitions — it starts quickly and slows down near the end of the animation to ease into the final value. It won't be ideal in every case, but it's generally a good choice.

That's better! Now we can see whether each bar is increasing or decreasing.



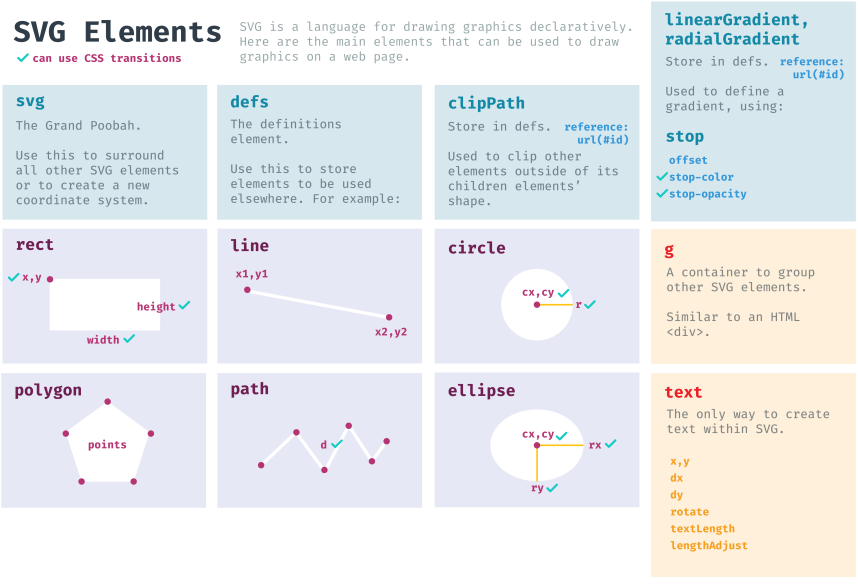
bars - slow change

Our transitions are still looking a bit disjointed with our text changing position instantaneously. Let's try to animate our text position, too.

```
.bin text {
    transition: y 1s ease-out;
}
```

Hmm, our text position is still not animating — it seems as if `y` isn't a transition-able property. Thankfully there is a workaround here — we can position the text using a CSS property instead of changing its `y` attribute.

Our advanced package has a handy SVG element cheat sheet with green check marks to show us what SVG elements' attributes are animate-able with CSS transitions. Keep it around for a quick reference when you're transitioning your own elements!



SVG elements cheat sheet

Switching over to our `updating-bars.js` file, let's position our bar labels using `translateY()`.

```
const barText = binGroups.select("text")
    .attr("x", d => xScale(d.x0) + (xScale(d.x1) - xScale(d.x0)) / 2)
    .attr("y", 0)
    .style("transform", d => `translateY(${
      yScale(yAccessor(d)) - 5
    }px)`)
    .text(d => yAccessor(d) || "")
```

> Note that we're filling our `<text>` elements with empty strings instead of `0` (with `.text(d => yAccessor(d) || "")`) to prevent labeling empty bars.

We'll also need to change the transition property to target `transform`.

```
.bin text {
    transition: transform 1s ease-out;
}
```

Now our bar labels are animating with our bars. Perfect!

Let's make one last change - we want our dashed mean line to animate when it moves left or right. We could try to `transition` changes to `x1` and `x2`, but those aren't CSS properties, they're SVG attributes. Let's position the line's horizontal position with the `transform` property.

```
const meanLine = bounds.selectAll(".mean")
    .attr("y1", -20)
    .attr("y2", dimensions.boundedHeight)
    .style("transform", `translateX(${xScale(mean)}px)`)
```

We'll also add the `transition` CSS property in our `styles.css` file:

```
.mean {
    stroke: maroon;
    stroke-dasharray: 2px 4px;
    transition: transform 1s ease-out;
}
```

These updates are looking great!

There are some animations that aren't possible with CSS transitions. For example, transitioning the x axis changes would help us see if the values for our new metric have increased or decreased. Using a CSS transition won't help here — CSS has no way of knowing whether a tick mark with the text `10` is larger than a tick mark with the text `100`. Let's bring out the heavier cavalry.

d3.transition

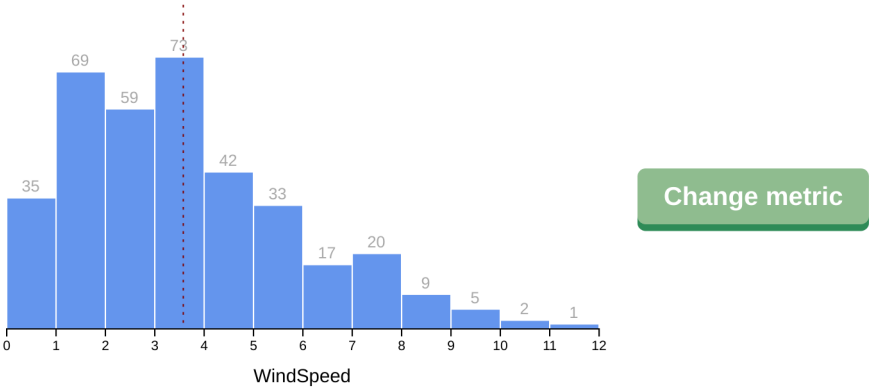
CSS transitions have our back for simple property changes, but for more complex animations we'll need to use `d3.transition()` from the **d3-transition**[26] module. When would we want to use `d3.transition()` instead of CSS transitions?

- When we want to ensure that multiple animations line up
- When we want to do something when the animation ends (for example starting another animation)
- When the property we want to animate isn't a CSS property (remember when we tried to animate our bars' heights but had to use `transform` instead? `d3.translate` can animate non-CSS property changes.)
- When we want to synchronize adding and removing elements with animations
- When we might interrupt halfway through a transition
- When we want a custom animation (for example, we could write a custom interpolator for changing text that adds new letters one-by-one)

Let's get our hands dirty by re-implementing the CSS transitions for our histogram. Navigate to the `/04-animations-and-transitions/3-draw-bars-with-d3-transition/`

[26] https://github.com/d3/d3-transition

folder — you'll see the same setup with our histogram and a big old **Change metric** button.



bars

Let's again start by animating any changes to our bars. Instead of adding a `transition` property to our `styles.css` file, we'll start in the `updating-bars.js` file where we set our `barRects` attributes.

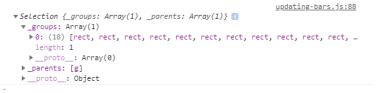
As a reminder, when we run:

```
const barRects = binGroups.select("rect")
```

we're creating a d3 selection object that contains all `<rect>` elements. Let's log that to the console as a refresher of what a selection object looks like.

```
const barRects = binGroups.select("rect")
        .attr("x", d => xScale(d.x0) + barPadding)
        .attr("y", d => yScale(yAccessor(d)))
        .attr("height", d => dimensions.boundedHeight
            - yScale(yAccessor(d))
        )
        .attr("width", d => d3.max([
          0,
          xScale(d.x1) - xScale(d.x0) - barPadding
        ]))
```

```
console.log(barRects)
```

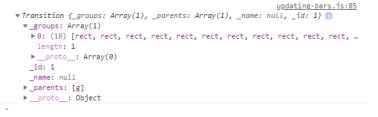


bars selection

We can use the `.transition()` method on our d3 selection object to transform our selection object into a d3 transition object.

```
const barRects = binGroups.select("rect")
    .transition()
        .attr("x", d => xScale(d.x0) + barPadding)
        .attr("y", d => yScale(yAccessor(d)))
        .attr("height", d => dimensions.boundedHeight
          - yScale(yAccessor(d))
        )
        .attr("width", d => d3.max([
          0,
          xScale(d.x1) - xScale(d.x0) - barPadding
        ]))

console.log(barRects)
```



bars transition

d3 transition objects look a lot like selection objects, with a `_groups` list of relevant DOM elements and a `_parents` list of ancestor elements. They have two additional keys: `_id` and `_name`, but that's not all that has changed.

Let's expand the `__proto__` of our transition object.

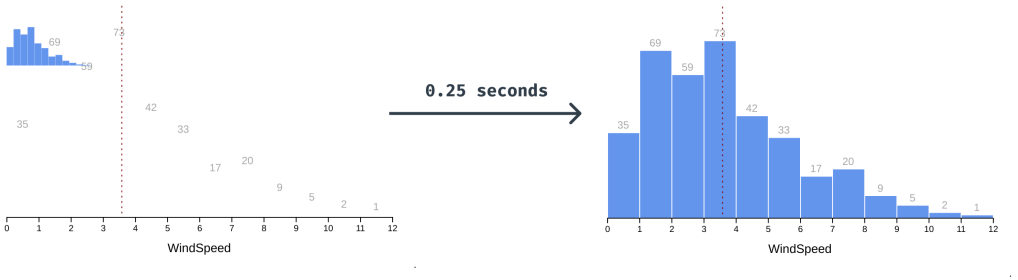
> `__proto__` is a native property of JavaScript objects that exposes methods and values that this specific object has inherited. If you're unfamiliar with JavaScript Prototypal Inheritance and want to read up, the MDN docs[a] are a good place to start.
>
> [a] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain

> In this case, we can see that the `__proto__` property contains d3-specific methods, and the nested `__proto__` object contains native object methods, such as `toString()`.

Transition object, expanded

We can see that some methods are inherited from d3 selection objects (eg. `.call()` and `.each()`), but most are overwritten by new transition methods. When we click the **Change metric** button now, we can see that our bar changes are animated. This makes sense — any `.attr()` updates chained after a `.transition()` call will use transition's `.attr()` method, which attempts to interpolate between old and new values.

Something looks strange though - our new bars are flying in from the top left corner.



Bars flying in

> Note that d3 transitions animate over 0.25 seconds — we'll learn how to change that in a minute!

Knowing that `<rect>`s are drawn in the top left corner by default, this makes sense. But how do we prevent this?

Remember how we can isolate new data points with `.enter()`? Let's find the line where we're adding new `<rect>`s and set their initial values. We want them to start in the right horizontal location, but be 0 pixels tall so we can animate them "growing" from the x axis.

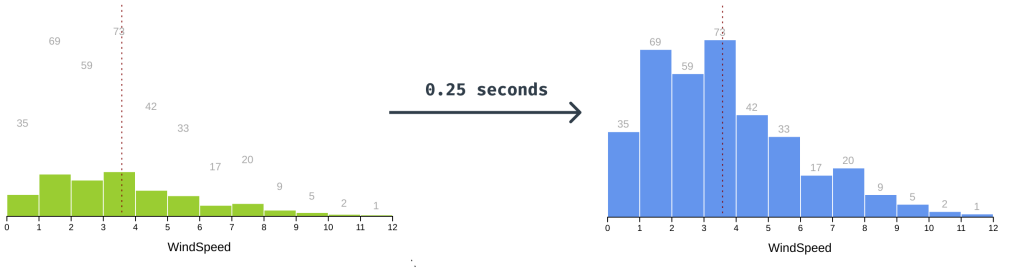
Let's also have them be green to start to make it clear which bars we're targeting. We'll need to set the fill using an inline style using `.style()` instead of setting the attribute in order to override the CSS styles in `styles.css`.

```
newBinGroups.append("rect")
        .attr("height", 0)
        .attr("x", d => xScale(d.x0) + barPadding)
        .attr("y", dimensions.boundedHeight)
        .attr("width", d => d3.max([
          0,
          xScale(d.x1) - xScale(d.x0) - barPadding
        ]))
        .style("fill", "yellowgreen")
```

> Why are we using `.style()` instead of `.attr()` to set the fill? We need the `fill` value to be an inline style instead of an SVG attribute in order to override the CSS styles in `styles.css`. The way CSS specificity works means that inline styles override class selector styles, which override SVG attribute styles.

Once our bars are animated in, they won't be new anymore. Let's transition their fill to blue. Luckily, chaining d3 transitions is really simple — to add a new transition that starts after the first one ends, add another `.transition()` call.

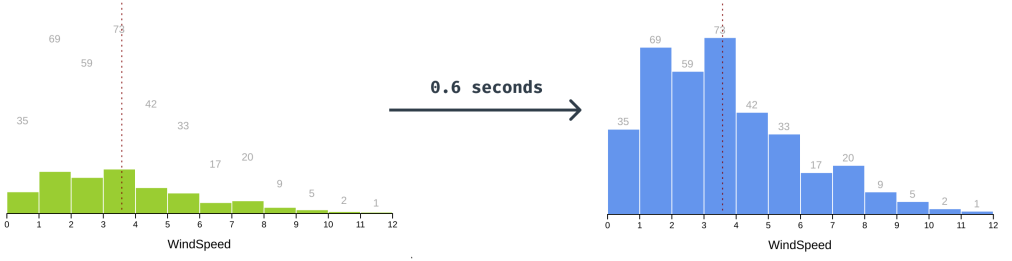
```
const barRects = binGroups.select("rect")
    .transition()
        .attr("x", d => xScale(d.x0) + barPadding)
        .attr("y", d => yScale(yAccessor(d)))
        .attr("height", d => dimensions.boundedHeight
          - yScale(yAccessor(d))
        )
        .attr("width", d => d3.max([
          0,
          xScale(d.x1) - xScale(d.x0) - barPadding
        ]))
    .transition()
        .style("fill", "cornflowerblue")
```



green bars on load

Let's slow things down a bit so we can bask in these fun animations. d3 transitions default to 0.25 seconds, but we can specify how long an animation takes by chaining `.duration()` with a number of milliseconds.

```
const barRects = binGroups.select("rect")
    .transition().duration(600)
        .attr("x", d => xScale(d.x0) + barPadding)
        // ...
```



green bars on load, 1 second

Smooth! Now that our bars are nicely animated, it's jarring when our text moves to its new position instantly. Let's add another transition to make our text transition with our bars.

```
const barText = binGroups.select("text")
    .transition().duration(600)
        .attr("x", d => xScale(d.x0)
          + (xScale(d.x1) - xScale(d.x0)) / 2
        )
        .attr("y", d => yScale(yAccessor(d)) - 5)
        .text(d => yAccessor(d) || "")
```

We'll also need to set our labels' initial position (higher up in our code) to prevent them from flying in from the left.

```
newBinGroups.append("text")
        .attr("x", d => xScale(d.x0)
          + (xScale(d.x1) - xScale(d.x0)) / 2
        )
        .attr("y", dimensions.boundedHeight)
```

Here's a fun tip: we can specify a timing function (similar to CSS's transition-timing-function) to give our animations some life. They can look super fancy, but we only need to chain `.ease()` with a d3 easing function. Check out the full list at the d3-ease repo[27].

```
const barRects = binGroups.select("rect")
    .transition().duration(600).ease(d3.easeBounceOut)
        .attr("x", d => xScale(d.x0) + barPadding)
        // ...
```

That's looking groovy, but our animation is out of sync with our labels again. We could ease our other transition, but there's an easier (no pun intended) way to sync multiple transitions.

By calling `d3.transition()`, we can make a transition on the root document that can be used in multiple places. Let's create a root transition — we'll need to place this definition above our existing transitions, for example after we define `barPadding`. Let's also log it to the console to take a closer look.

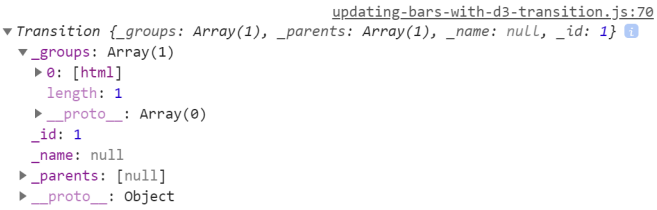
```
const barPadding = 1

const updateTransition = d3.transition()
    .duration(600)
    .ease(d3.easeBackIn)

console.log(updateTransition)
```

If we expand the `_groups` array, we can see that this transition is indeed targeting our root `<html>` element.

[27] https://github.com/d3/d3-ease



Root transition object

> You'll notice errors in the dev tools console that say `Error: <rect> attribute height: A negative value is not valid.`. This happens with the `d3.easeBackIn` easing we're using, which causes the bars to bounce below the x axis when they animate.

Let's update our bar transition to use `updateTransition` instead of creating a new transition. We can do this by passing the existing transition in our `.transition()` call.

```
const barRects = binGroups.select("rect")
    .transition(updateTransition)
        .attr("x", d => xScale(d.x0) + barPadding)
        // ...
```

Let's use `updateTransition` for our text, too.

```
const barText = binGroups.select("text")
    .transition(updateTransition)
        .attr("x", d => xScale(d.x0)
          + (xScale(d.x1) - xScale(d.x0)) / 2
        )
        // ...
```

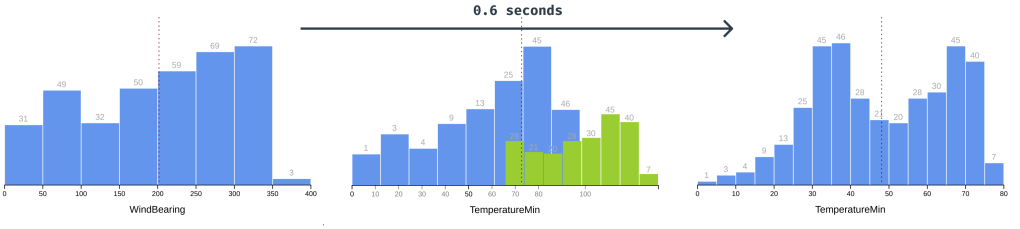
We can use this transition as many times as we want — let's also animate our mean line when it updates.

```
const meanLine = bounds.selectAll(".mean")
    .transition(updateTransition)
        .attr("x1", xScale(mean))
        // ...
```

And our x axis:

```
const xAxis = bounds.select(".x-axis")
    .transition(updateTransition)
    .call(xAxisGenerator)
```

Remember that we couldn't animate our x axis with CSS transition? Our transition objects are built to handle axis updates — we can see our tick marks move to fit the new **domain** before the new tick marks are drawn.



Bars - transition all elements

But what about animating our bars when they leave? Good question - exit animations are often difficult to implement because they involve delaying element removal. Thankfully, d3 transition makes this pretty simple.

Let's start by creating a transition right before we create `updateTransition`. Let's also take out the easing we added to `updateTransition` since it's a little distracting.

```
const exitTransition = d3.transition().duration(600)
const updateTransition = d3.transition().duration(600)
```

We can target only the bars that are exiting using our `.exit()` method. Let's turn them red before they animate to make it clear which bars are leaving. Then we can use our `exitTransition` and animate the `y` and `height` values so the bars shrink into the x axis.

> Don't look at the browser just yet, we'll need to finish our exit transition first.

```
const oldBinGroups = binGroups.exit()
oldBinGroups.selectAll("rect")
        .style("fill", "red")
    .transition(exitTransition)
        .attr("y", dimensions.boundedHeight)
        .attr("height", 0)
```

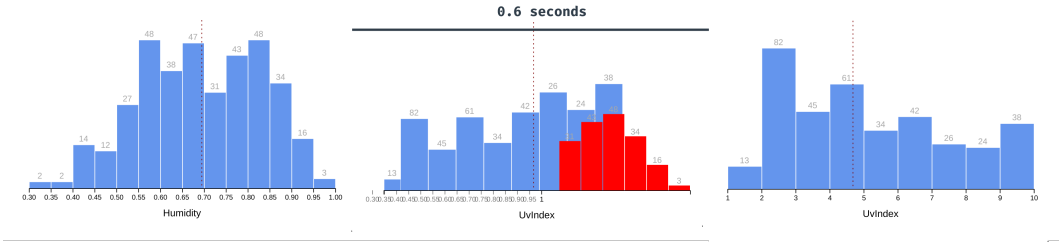
And we'll remember to also transition our text this time:

```
oldBinGroups.selectAll("text")
  .transition(exitTransition)
      .attr("y", dimensions.boundedHeight)
```

Last, we need to actually remove our bars from the DOM. We'll use a new transition here — not because we can animate removing the elements, but to delay their removal until the transition is over.

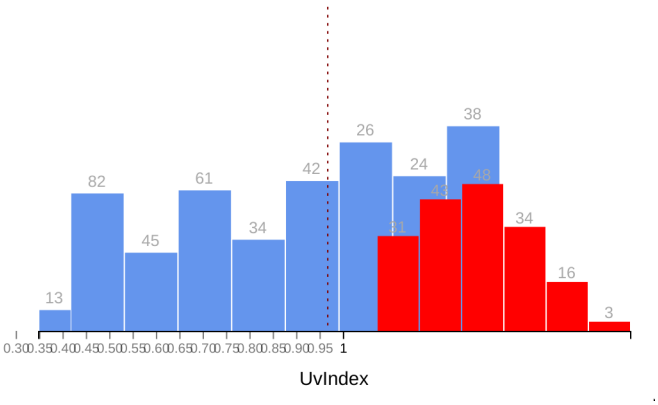
```
oldBinGroups
    .transition(exitTransition)
        .remove()
```

Now we can look at the browser, and we can see our bars animating in and out!



Bars - transition in and out

There is one issue, though: our bars are moving to their new positions while the bars are still exiting and we end up with intermediate states like this one:



Transition - intermediate

To fix this, we'll delay the update transition until the exit transition is finished. Instead of creating a our `updateTransition` as a root transition, we can chain it on our existing `exitTransition`.

```
const exitTransition = d3.transition().duration(600)
const updateTransition = exitTransition.transition().duration(600)
```

> We're chaining transitions here to run them one after the other — d3 transitions also have a `.delay()` method if you need to delay a transition for a certain amount of time. Check out the docs[a] for more information.
>
> [a] https://github.com/d3/d3-transition#transition_delay

Wonderful! Now that we've gone through the three different ways we can animate changes, let's recap when each method is appropriate.

SVG **`<animate>`** is only appropriate for static animations.

CSS **`transition`** is useful for animating CSS properties. A good rule of thumb is to use these mainly for stylistic polish — that way we can keep simpler transitions in our stylesheets, with the main goal of making our visualizations feel smoother.

`d3.transition()` is what we want to use for more complex animations: whenever we need to chain or synchronize with another transition or with DOM changes.

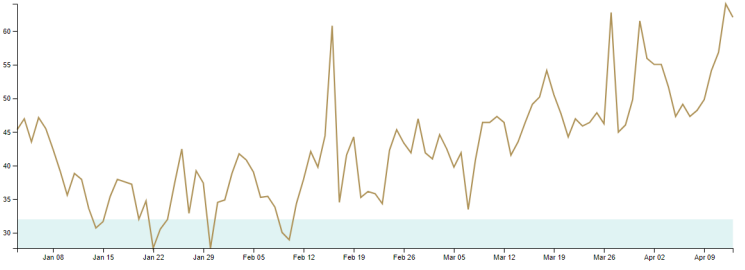
Lines

After animating bars, animating line transitions should be easy, right? Let's find out!

Let's navigate to the `/04-animations-and-transitions/4-draw-line/` folder and open the `updating-line.js` file.

Look familiar? This is our timeline drawing code from **Chapter 1** with some small updates.

> You might need to update your "freezing" temperatures, if you live in a warm place and are getting errors in the console.



Timeline

One of the main changes is an `addNewDay()` function at the bottom of the script. This exact code isn't important — what is good to know is that `addNewDay()` shifts our dataset one day in the future. To simulate a live timeline, `addNewDay()` runs every 1.5 seconds.

> If you read through the `addNewDay()` code and were confused by the `...dataset.slice(1),` syntax, the `...` is using ES6 spread syntax to expand the dataset (minus the first

point) in place. Read more about it in the MDN docs[a].

[a] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax

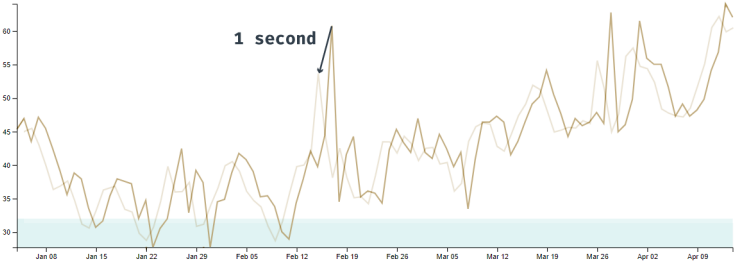
We can see our timeline updating when we load our webpage, but it looks jerky. We know how to smooth the axis transitions, let's make them nice and slow.

```
const xAxis = bounds.select(".x-axis")
    .transition().duration(1000)
    .call(xAxisGenerator)
```

Great! Now let's transition the line.

```
const line = bounds.select(".line")
    .transition().duration(1000)
        .attr("d", lineGenerator(dataset))
```

What's going on here? Why is our line wriggling around instead of adding a new point at the end?



Timeline wriggling?

Remember when we talked about how path **d** attributes are a string of draw-to values, like a learn-coding turtle? d3 is transitioning each point to **the next point at the same index**. Our transition's `.attr()` function has no idea that we've just shifted our points down one index. It's guessing how to transition to the new **d** value, animating each point to the next day's y value.

Pretend you're the `.attr()` function - how would you transition between these two **d** values?

```
<path d= "M 0 50 L 1 60 L 2 70 L 3 80 Z" />
<path d= "M 0 60 L 1 70 L 2 80 L 3 90 Z" />
```

It would make the most sense to transition each point individually, interpolating from `0 50` to `0 60` instead of moving each point to the left.

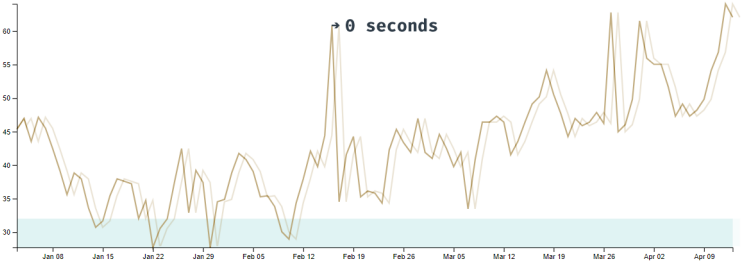
Great, we understand *why* our line is wriggling, but how do we shift it to the left instead?

Let's start by figuring out **how far we need to shift our line** to the left. Before we update our line, let's grab the last two points in our dataset and find the difference between their x values.

```
const lastTwoPoints = dataset.slice(-2)
const pixelsBetweenLastPoints = xScale(xAccessor(lastTwoPoints[1]))
  - xScale(xAccessor(lastTwoPoints[0]))
```

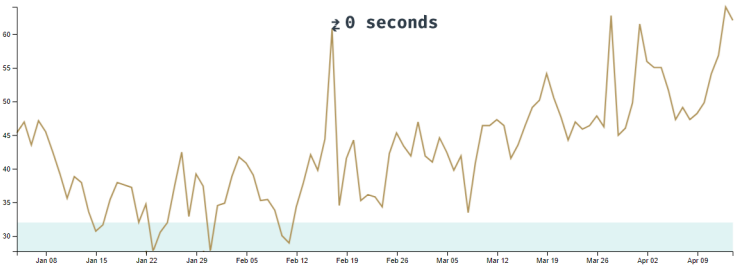
Now when we update our line, we can instantly shift it to the right to match the old line.

```
const line = bounds.select(".line")
        .attr("d", lineGenerator(dataset))
        .style("transform", `translateX(${pixelsBetweenLastPoints}px)`)
```



Timeline shifting to right

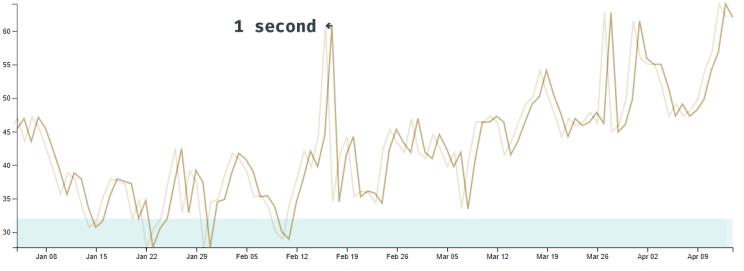
This shift should be invisible because **at the same time we're shifting our x scale to the left by the same amount**.



Timeline shifting to right then the left

Then we can animate un-shifting the line to the left, to its normal position on the x axis.

```
const line = bounds.select(".line")
        .attr("d", lineGenerator(dataset))
        .style("transform", `translateX(${pixelsBetweenLastPoints}px)`)
    .transition().duration(1000)
        .style("transform", `none`)
```



Timeline updating

Okay great! We can see the line updating before it animates to the left, but we don't want to see the new point until it's within our bounds. **The easiest way to hide out-of-bounds data is to add a `<clipPath>`.**

A `<clipPath>` is an SVG element that:

- is sized by its children. If a `<clipPath>` contains a **circle**, it will only paint content within that circle's bounds.
- can be referenced by another SVG element, using the `<clipPath>`'s `id`.

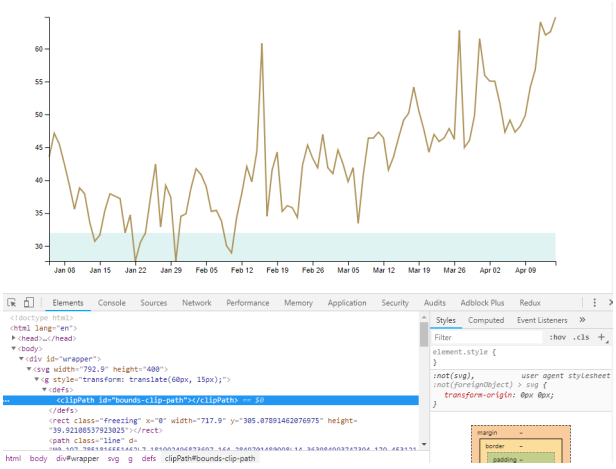
Before we test it out, we need to learn one important SVG convention: using `<defs>`. The SVG `<defs>` element is used to store any re-usable definitions that are used later in the `<svg>`. By placing any `<clipPath>`s or gradients in our `<defs>` element, we'll make our code more accessible. We'll also know where to look when we're debugging, similar to defining constants in one place before we use them.

Now that we know this convention, let's create our `<defs>` element and add our `<clipPath>` inside. We'll want to put this definition right after we define our **bounds**. Let's also give it an `id` that we can reference later.

```
const bounds = wrapper.append("g")
    .style("transform", `translate(${
      dimensions.margin.left
    }px, ${
      dimensions.margin.top
    }px)`)

bounds.append("defs")
  .append("clipPath")
    .attr("id", "bounds-clip-path")
```

If we inspect our `<clipPath>` in the Elements panel, we can see that it's not rendering at all.



clipPath

Remember, the `<clipPath>` element's shape depends on its children, and it has no children yet. Let's add a `<rect>` that covers our bounds.

```
bounds.append("defs")
  .append("clipPath")
    .attr("id", "bounds-clip-path")
  .append("rect")
    .attr("width", dimensions.boundedWidth)
    .attr("height", dimensions.boundedHeight)
```

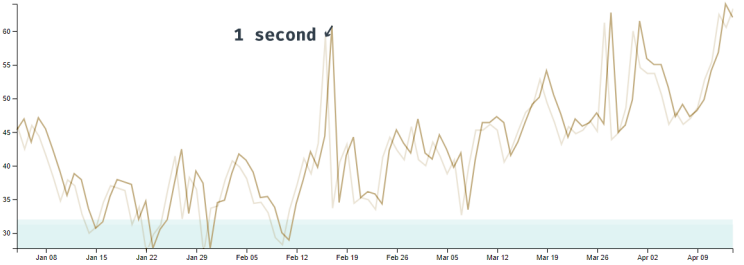
To use our `<clipPath>` we'll create a group with the attribute `clip-path` pointing to our `<clipPath>`'s `id`. The order in which we draw SVG elements determines their "z-index". Keeping that in mind, let's add our new group *after* we draw the **freezing** `<rect>`.

```
bounds.append("rect")
    .attr("class", "freezing")
const clip = bounds.append("g")
    .attr("clip-path", "url(#bounds-clip-path)")
```

Now we can update our `path` to sit inside of our new group, instead of the `bounds`.

```
clip.append("path")
    .attr("class", "line")
```

Voila! When we reload our webpage, we can see that our line's new point isn't fully visible until it has finished un-shifting.



Timeline transition

> We can see that the first point of our dataset is being removed before our line un-shifts. I bet you could think of a few ways around this — feel free to implement one or two! We could save the old dataset and preserve that extra point until our line is unshifted, or we could slice off the first data point when we define our x scale. In a production graph, the solution would depend on how our data is updating and what's appropriate to show.

Now that we have the tools needed to make our chart transitions lively, we'll learn how to let our users interact with our charts!

Interactions

The biggest advantage of creating charts with JavaScript is the ability to respond to user input. In this chapter, we'll learn what ways users can interact with our graphs and how to implement them.

d3 events

Browsers have native **event listeners** — using `addEventListener()`, we can listen for events from a user's:

- mouse
- keyboard
- scroll wheel
- touch
- resize
- ... and more.

For example:

```
function onClick(event) {
  // do something here...
}
addEventListener("click", onClick)
```

After running this code, the browser will trigger `onClick()` when a user clicks anywhere on the page.

These event listeners have tons of functionality and are simple to use. We can get even more functionality using d3's event listener wrappers!

Our d3 selection objects have an `.on()` method that will create event listeners on our selected DOM elements. Let's take a look at how to implement d3 event listeners. First, we'll need to get our server up and running `live-server` and navigate to `/code/05-interactions/1-events/draft/`.

If we open the `events.js` file, we can see a few things happening:

1. We define `rectColors` as an array of colors.
2. We grab all `.rect` elements inside of the `#svg` element (created in `index.html`) and bind our selection to the `rectColors` array.
3. We use `.enter()` to isolate all new data points (every row in `rectColors`) and `.append()` a `<rect>` for each row.
4. Lastly, we set each `<rect>`'s size to 100 pixels by 100 pixels and shift each item 110 pixels to the right (multiplied by its index). We also make all of our boxes light grey.

In our browser, we can see our four boxes.

grey boxes

They don't do much right now, let's make it so they change to their designated color on hover.

To add a d3 event listener, we pass the type of event we want to listen for as the first parameter of `.on()`. Any DOM event type will work — see the full list of event types on the MDN docs[28]. To mimic a hover start, we'll want to target `mouseenter`.

```
rects.on("mouseenter")
```

The second parameter `.on()` receives is a callback function that will be executed when the specified event happens. This function will receive three parameters:

[28] https://developer.mozilla.org/en-US/docs/Web/Events#Standard_events

1. the *bound datum*
2. the current *index*
3. the *currently selected nodes*

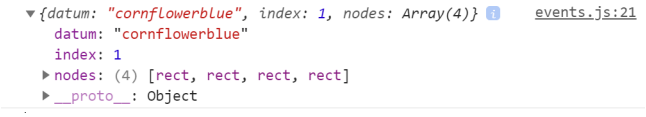
Let's log these parameters to the console to get a better look.

```
rects.on("mouseenter", function(datum, index, nodes) {
  console.log({datum, index, nodes})
})
```

It can often be helpful to use ES6 object property shorthand for logging multiple variables. This way, we can see the name and value of each variable!

When we hover over a box, we can see that our `mouseenter` event is triggered! The parameters passed to our function(in order) are:

1. the matching *data point* bound from the `rectColors` array (in this case, the color)
2. the rect's *index*
3. the *nodes* in the current selection (in this case, the list of `<rect>` s)

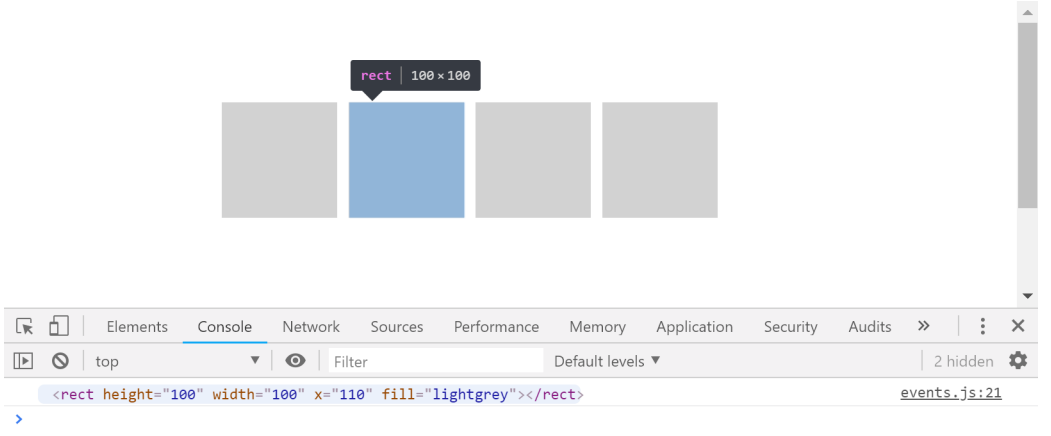


function parameters

In order to change the color of the current box, we'll need to create a d3 selection targeting only that box. We *could* find it in the list of nodes using its index, but there's an easier way. Let's take a look at what `this` looks like in our function.

```
rects.on("mouseenter", function(datum, index, nodes) {
  console.log(this)
})
```

Perfect! It looks like the `this` keyword points at the DOM element that triggered the event.



function this

We can use `this` to create a d3 selection and set the box's fill using the `datum`.

```
rects.on("mouseenter", function(datum, index, nodes) {
  d3.select(this).style("fill", datum)
})
```

Now when we refresh our webpage, we can change our boxes to their related color on hover!

hovered boxes

Hmm, we're missing something. We want our boxes to turn back to grey when our mouse leaves them. Let's chain another event listener that triggers on `mouseout` and make our box grey again.

```
rects.on("mouseenter", function(datum, index, nodes) {
  d3.select(this).style("fill", datum)
})
.on("mouseout", function() {
  d3.select(this).style("fill", "lightgrey")
})
```

`mouseenter` is often seen as interchangeable with `mouseover`. The two events are very similar, but `mouseenter` is usually closer to the wanted behavior. They are both triggered when the mouse enters the targeted container, but `mouseover` is also triggered when the mouse moves between nested elements.

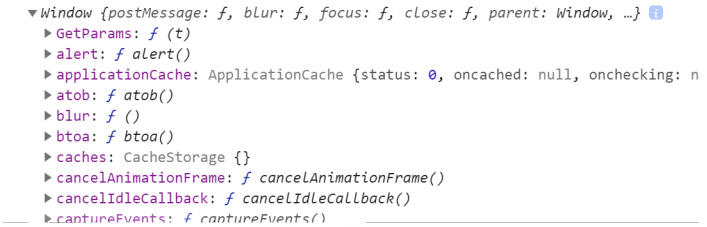
This same distinction applies to `mouseleave` (preferred) and `mouseout`.

Don't use fat arrow functions

You might be wondering why we're not using ES6 fat arrow functions here. Let's look at what happens to our `this` keyword when we use an arrow function:

```
rects.on("mouseenter", () => {
  console.log(this)
})
```

Oh right, `this` in arrow functions refer to the *lexical scope*, meaning that `this` will always refer to the same thing inside and outside of a function.



function this (arrow)

This is why we use normal `function() {}` declarations when creating d3 event listeners, so we can access the targeted element with `this`.

Destroying d3 event listeners

Before we look at adding events to our charts, let's learn how to destroy our event handlers. Removing old event listeners is important for updating charts and preventing memory leaks, among other things.

Let's add a 3 second timeout at the end of our code so we can test that our mouse events are working before we destroy them.

```
setTimeout(() => {
}, 3000)
```

Removing a d3 event listener is easy — all we need to do is call `.on()` with `null` as the triggered function.

```
setTimeout(() => {
    rects
        .on("mouseenter", null)
        .on("mouseout", null)
}, 3000)
```

Perfect! Now our hover events will stop working after 3 seconds. You might have noticed that a box might be stuck with its hovered color if it was hovered over when the mouse events were deleted.

hovered boxes - stuck!

Luckily, there's an easy fix!

D3 selections have a `.dispatch()` method that will programatically trigger an event — no mouseout needed. We can trigger a `mouseout` event right before we remove it to ensure that our boxes finish in their "neutral" state.

```
setTimeout(() => {
    rects
        .dispatch("mouseout")
        .on("mouseenter", null)
        .on("mouseout", null)
}, 3000)
```

Perfect! Now that we have a good handle on using d3 event listeners, let's use them to make our charts interactive.

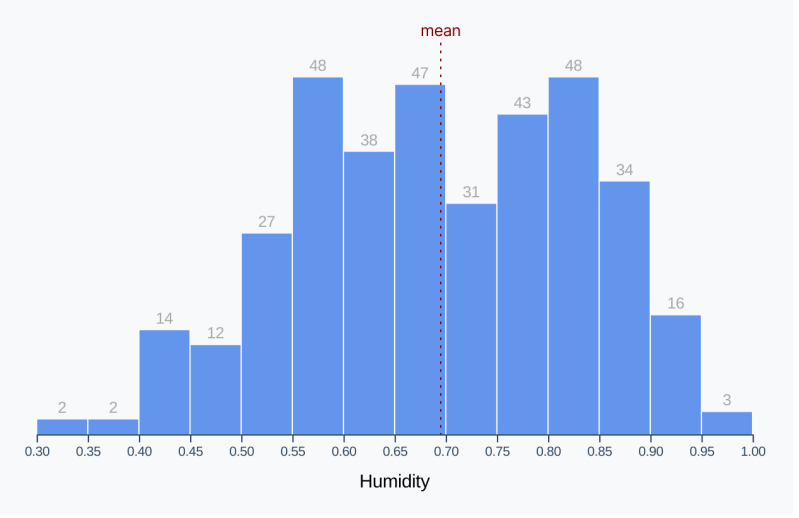
Bar chart

Let's add interactions to our histogram — navigate to

`/code/05-interactions/2-bars/draft/` in the browser and open the

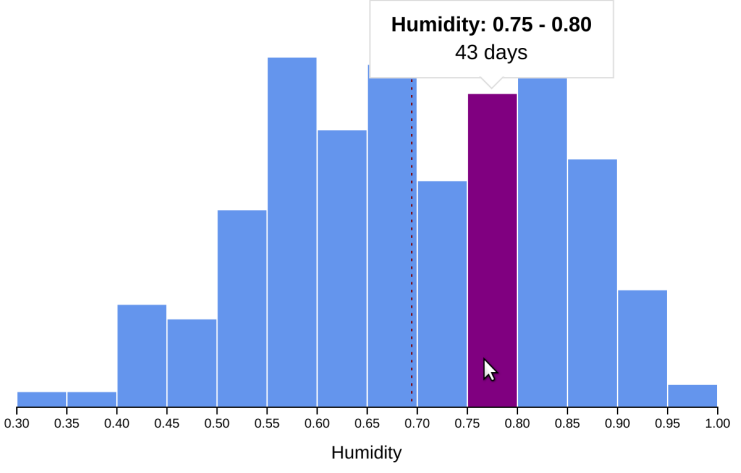
`/code/05-interactions/2-bars/draft/bars.js` file in your text editor.

We should see the histogram we created in **Chapter 3**.



Histogram

Our goal in the section is to add an informative tooltip that shows the humidity range and day count when a user hovers over a bar.



histogram finished

We could use d3 event listeners to change the bar's color on hover, but there's an alternative: CSS hover states. To add CSS properties that only apply when an element

is hovered over, add `:hover` after the selector name. It's good practice to place this selector immediately after the non-hover styles to keep all bar styles in one place.

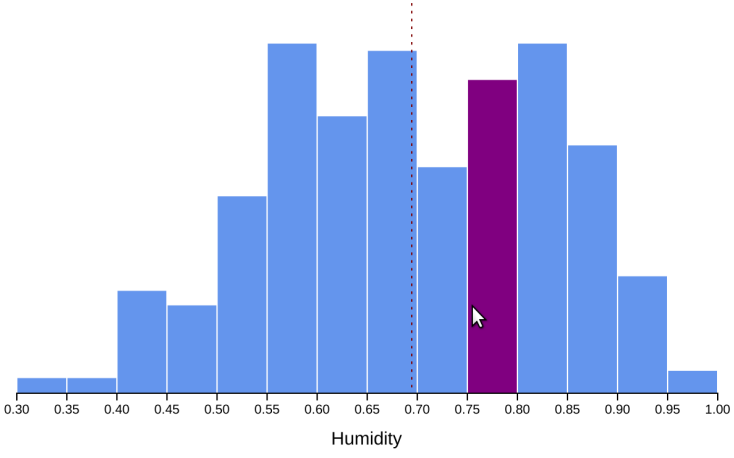
Let's add a new selector to the `/code/05-interactions/2-bars/draft/styles.css` file.

```
.bin rect:hover {
}
```

Let's have our bars change their fill to purple when we hover over them.

```
.bin rect:hover {
    fill: purple;
}
```

Great, now our bars should turn purple when we hover over them and back to blue when we move our mouse out.



histogram with hover state

Now we know how to implement hover states in two ways: CSS hover states and event listeners. Why would we use one over the other?

CSS hover states are good to use for more *stylistic* updates that don't require DOM changes. For example, changing colors or opacity. If we're using a CSS preprocessor like SASS, we can use any color variables instead of duplicating them in our JavaScript file.

JavaScript event listeners are what we need to turn to when we need a more complicated hover state. For example, if we want to update the text of a tooltip or move an element, we'll want to do that in JavaScript.

Since we need to update our tooltip text and position when we hover over a bar, let's add our `mouseenter` and `mouseleave` event listeners at the bottom of our `bars.js` file. We can set ourselves up with named functions to keep our chained code clean and concise.

```
binGroups.select("rect")
    .on("mouseenter", onMouseEnter)
    .on("mouseleave", onMouseLeave)

function onMouseEnter(datum) {
}

function onMouseLeave(datum) {
}
```

Starting with our `onMouseEnter()` function, we'll start by grabbing our tooltip element. If you look in our `index.html` file, you can see that our template starts with a tooltip with two children: a div to display the range and a div to display the value. We'll follow the common convention of using **id**s as hooks for JavaScript and **class**es as hooks for CSS. There are two main reasons for this distinction:

1. We can use classes in multiple places (if we wanted to style multiple elements at once) but we'll only use an id in one place. This ensures that we're selecting the correct element in our chart code
2. We want to separate our chart manipulation code and our styling code — we should be able to move our chart hook without affecting the styles.

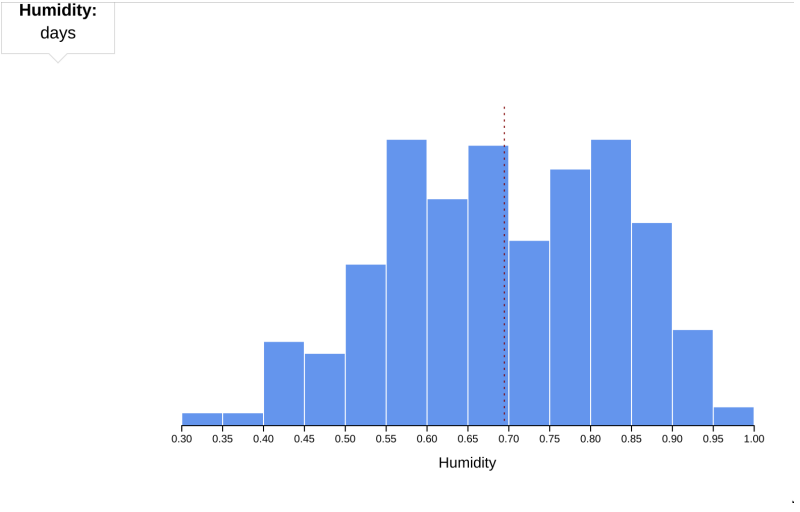
> We could create our tooltip in JavaScript, the same way we have been creating and manipulating SVG elements with d3. We have it defined in our HTML file here, which is generally easier to read and maintain since the tooltip layout is static.

If we open up our **styles.css**, we can see our basic tooltip styles, including using a pseudo-selector `.tooltip:before` to add an arrow pointing down (at the hovered bar). Also note that the tooltip is hidden (`opacity: 0`) and will transition any property changes (`transition: all 0.2s ease-out`). It also will not receive any mouse events (`pointer-events: none`) to prevent from stealing the mouse events we'll be implementing.

Let's comment out the `opacity: 0` property so we can get a look at our tooltip.

```
.tooltip {
    /* opacity: 0; */
```

We can see that our tooltip is positioned in the top left of our page.



histogram with visible tooltip - far away!

If we position it instead at the top left of our chart, we'll be able to shift it based on the hovered bar's position in the chart.

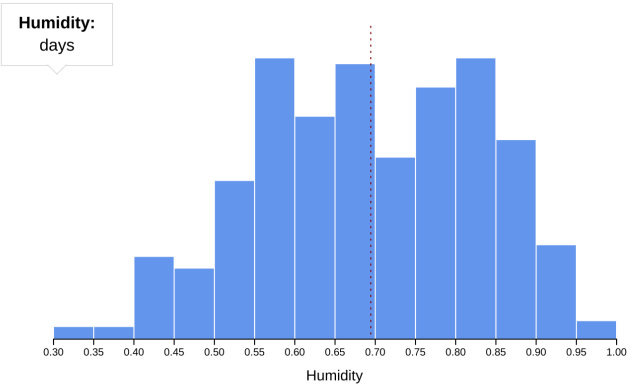
We can see that our tooltip is absolutely positioned all the way to the left and 12px above the top (to offset the bottom triangle). So why isn't it positioned at the top left of our chart?

Absolutely positioned elements are placed relative to their **containing block**. The default containing block is the `<html>` element, but will be overridden by certain ancestor elements. The main scenario that will create a new containing block is if the element has a `position` other than the default (`static`). There are other scenarios, but they are much more rare (for example, if a `transform` is specified).

This means that our tooltip will be positioned at the top left of the nearest ancestor element that has a set `position`. Let's give our `.wrapper` element a position of `relative`.

```
.wrapper {
    position: relative;
}
```

Perfect! Now our tooltip is located at the top left of our chart and ready to be shifted into place when a bar is hovered over.



histogram with visible tooltip

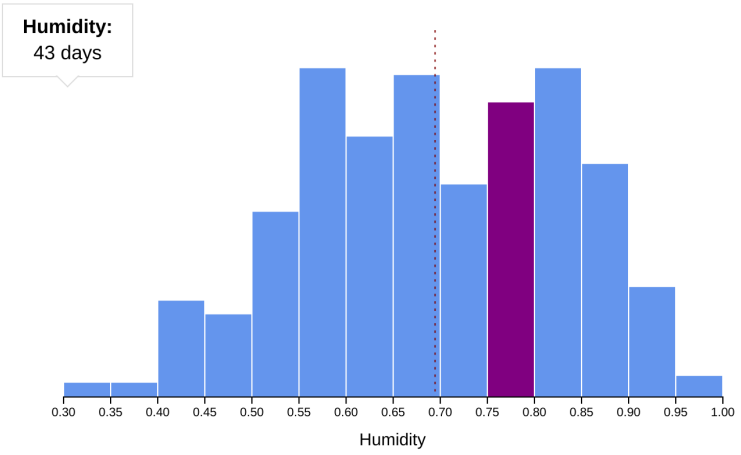
Let's start adding our mouse events in `bars.js` by grabbing the existing tooltip using its id (`#tooltip`). Our tooltip won't change once we load the page, so let's define it outside of our `onMouseEnter()` function.

```
const tooltip = d3.select("#tooltip")
function onMouseEnter(datum) {
}
```

Now let's start fleshing out our `onMouseEnter()` function by updating our tooltip text to tell us about the hovered bar. Let's select the nested `#count` element and update it to display the y value of the bar. Remember, in our histogram the y value is the number of days in our dataset that fall in that humidity level range.

```
const tooltip = d3.select("#tooltip")
function onMouseEnter(datum) {
    tooltip.select("#count")
        .text(yAccessor(datum))
}
```

Looking good! Now our tooltip updates when we hover over a bar to show that bar's count.

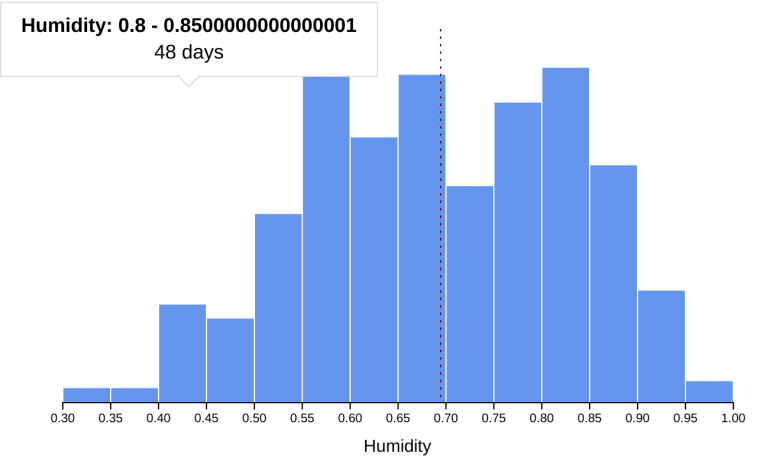


histogram tooltip with count

Next, we can update our range value to match the hovered bar. The bar is covering a range of humidity values, so let's make an array of the values and join them with a - (which can be easier to read than a template literal).

```
tooltip.select("#range")
    .text([
        datum.x0,
        datum.x1
    ].join(" - "))
```

Our tooltip now updates to display both the count *and* the range, but it might be a bit *too* precise.



histogram tooltip with count and range

We could convert our range values to strings and slice them to a certain precision, but there's a better way. It's time to meet `d3.format()`.

The **d3-format**[29] module helps turn numbers into nicely formatted strings. Usually when we display a number, we'll want to parse it from its raw format. For example, we'd rather display `32,000` than `32000` — the former is easier to read and will help with scanning a list of numbers.

If we pass `d3.format()` a **format specifier string**, it will create a formatter function. That formatter function will take one parameter (a number) and return a formatted string. There are many possible format specifier strings — let's go over the format for the options we'll use the most often.

```
[,][.precision][type]
```

Each of these specifiers is optional — if we use an empty string, our formatter will just return our number as a string. Let's talk about what each specifier tells our formatter.

`,`: add commas every 3 digits to the left of the decimal place

`.precision`: give me this many numbers after the decimal place.

`type`: each specific type is declared by using a single letter or symbol. The most handy types are:

[29] https://github.com/d3/d3-format

- **f**: fixed point notation — give me `precision` many decimal points
- **r**: decimal notation — give me `precision` many significant digits and pad the rest until the decimal point
- **%**: percentage — multiply my number by 100 and return `precision` many decimal points

Run through a few examples in your terminal to get the hang of it.

```
d3.format( ".2f")(11111.111) // "11111.11"
d3.format(",.2f")(11111.111) // "11,111.11"
d3.format(",.0f")(11111.111) // "11,111"
d3.format(",.4r")(11111.111) // "11,110"
d3.format( ".2%")(0.111)     // "11.10%"
```

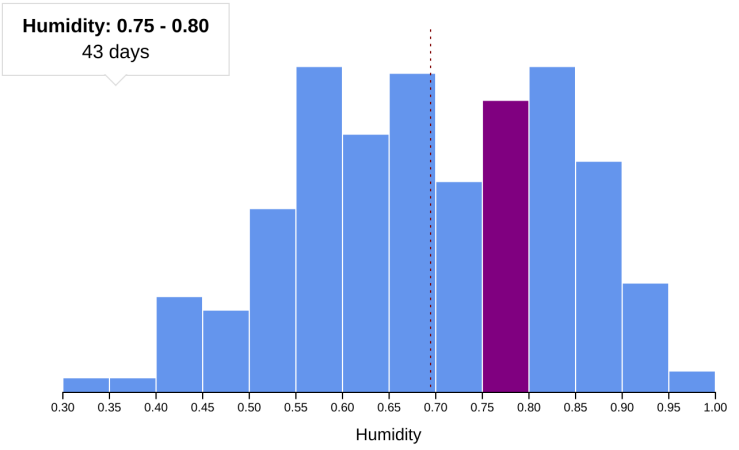
Let's create a formatter for our humidity levels. Two decimal points should be enough to differentiate between ranges without overwhelming our user with too many `0`s.

```
const formatHumidity = d3.format(".2f")
```

Now we can use our formatter to clean up our humidity level numbers.

```
const formatHumidity = d3.format(".2f")
tooltip.select("#range")
    .text([
        formatHumidity(datum.x0),
        formatHumidity(datum.x1)
    ].join(" - "))
```

Nice! An added benefit to our number formatting is that our range numbers are the same width for every value, preventing our tooltip from jumping around.



histogram tooltip with count and formatted range

Next, we want to position our tooltip *horizontally centered* above a bar when we hover over it. To calculate our tooltip's `x` position, we'll need to take three things into account:

- the bar's `x` position in the chart (`xScale(datum.x0)`),
- half of the bar's **width** ((xScale(datum.x1) - xScale(datum.x0)) / 2‘), and
- the **margin** by which our **bounds** are shifted **right** (`dimensions.margin.left`).

Remember that our tooltip is located at the top left of our **wrapper** - the outer container of our chart. But since our bars are within our **bounds**, they are shifted by the margins we specified.

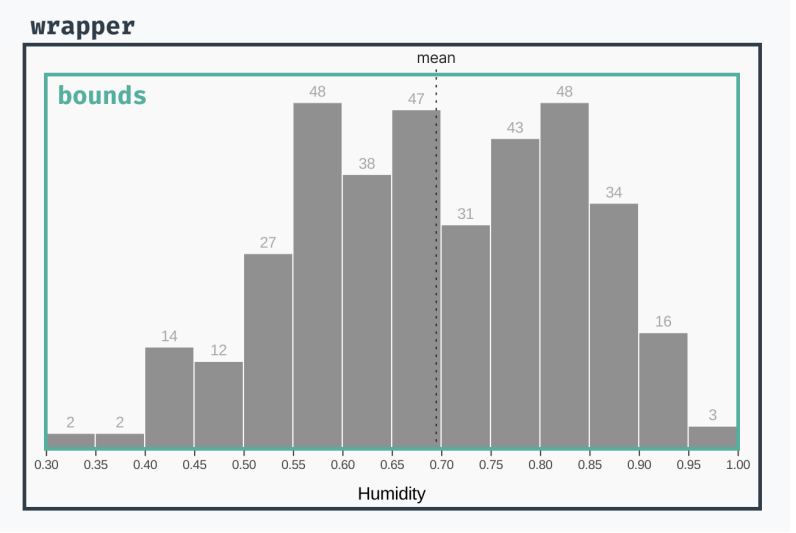


Chart terminology

Let's add these numbers together to get the `x` position of our tooltip.

```
const x = xScale(datum.x0)
  + (xScale(datum.x1) - xScale(datum.x0)) / 2
  + dimensions.margin.left
```

When we calculate our tooltip's y position, we don't need to take into account the bar's dimensions because we want it placed above the bar. That means we'll only need to add two numbers:

1. the bar's `y` position (`yScale(yAccessor(datum))`), and
2. the **margin** by which our **bounds** are shifted **down** (`dimensions.margin.top`)

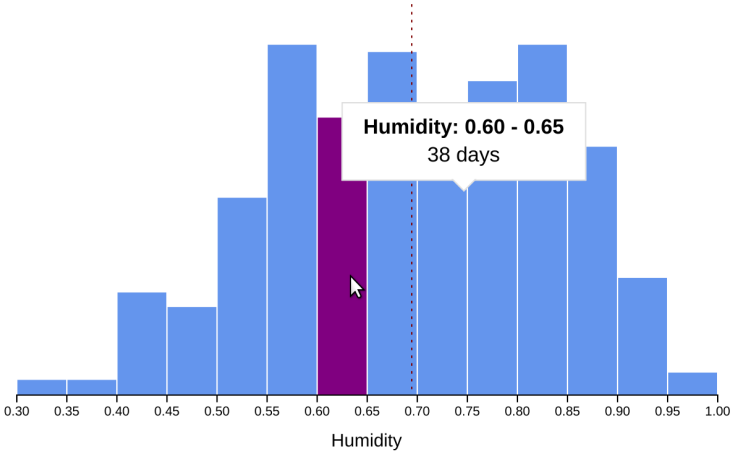
```
const y = yScale(yAccessor(datum))
  + dimensions.margin.top
```

Let's use our `x` and `y` positions to shift our tooltip. Because we're working with a normal xHTML **div**, we'll use the CSS **translate** property.

```
tooltip.style("transform", `translate(`
  + `${x}px,`
  + `${y}px`
  + `)`)
```

> Why are we setting the **transform** CSS property and not **left** and **top**? A good rule of thumb is to avoid changing (and especially animating) CSS values other than **transform** and **opacity**. When the browser styles elements on the page, it runs through several steps:
>
> 1. calculate style
> 2. layout
> 3. paint, and
> 4. layers
>
> Most CSS properties affect steps 2 or 3, which means that the browser has to perform that step and the subsequent steps every time that property is changed. **Transform** and **opacity** only affect step 4, which cuts down on the amount of work the browser has to do. Read more about each step and this distinction at https://www.html5rocks.com/en/tutorials/speed/high-performance-animations/[a].
>
> [a] https://www.html5rocks.com/en/tutorials/speed/high-performance-animations/

Hmm, why is our tooltip in the wrong position? It looks like we're positioning the top left of the tooltip in the right location (above the hovered bar).



histogram tooltip, unshifted

We want to position the **bottom, center** of our tooltip (the tip of the arrow) above the bar, instead. We *could* find the tooltip size by calling the `.getBoundingClientRect()` method, but there's a computationally cheaper way.

There are a few ways to shift absolutely positioned elements using CSS properties:

- `top`, `left`, `right`, and `bottom`
- `margins`
- `transform: translate()`

All of these properties can receive percentage values, but some of them are based on different dimensions.

- `top` and `bottom`: **percentage of the parent's height**
- `left` and `right`: **percentage of the parent's width**
- `margins`: **percentage of the parent's width**
- `transform: translate()`: **percentage of the specified element**

We're interested in shifting the tooltip based on its own height and width, so we'll need to use `transform: translate()`. But we're *already* applying a `translate` value — **how can we set the `translate` value using a pixel amount and a width**?

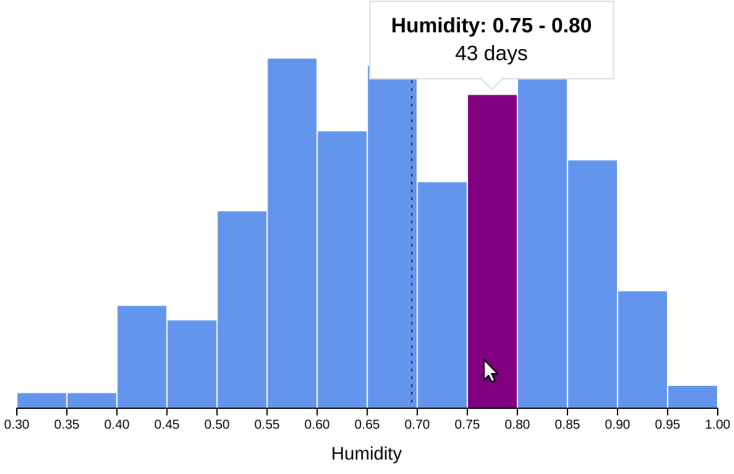
CSS `calc()` comes to the rescue here! We can tell CSS to calculate an offset based on values with different units. For example, the following CSS rule would cause an element to be 20 pixels wider than its container.

```
width: calc(100% + 20px);
```

Let's use `calc()` to offset our tooltip **up** half of its own width (`-50%`) and **left** `-100%` of its own height. This is in addition to our calculated `x` and `y` values.

```
tooltip.style("transform", `translate(`
  + `calc( -50% + ${x}px),`
  + `calc(-100% + ${y}px)`
  + `)`)
```

Perfect! Now our tooltip moves to the exact location we want.



histogram finished

We have one last task to do — hide the tooltip when we're not hovering over a bar. Let's un-comment the `opacity: 0` rule in `styles.css` so its hidden to start.

```
.tooltip {
    opacity: 0;
```

Jumping back to our bars.js file, we need to make our tooltip visible at the end of our `onMouseEnter()` function.

```
tooltip.style("opacity", 1)
```

Lastly, we want to make our tooltip invisible again whenever our mouse leaves a bar. Let's add that to our `onMouseLeave()` function.

```
function onMouseLeave() {
  tooltip.style("opacity", 0)
}
```

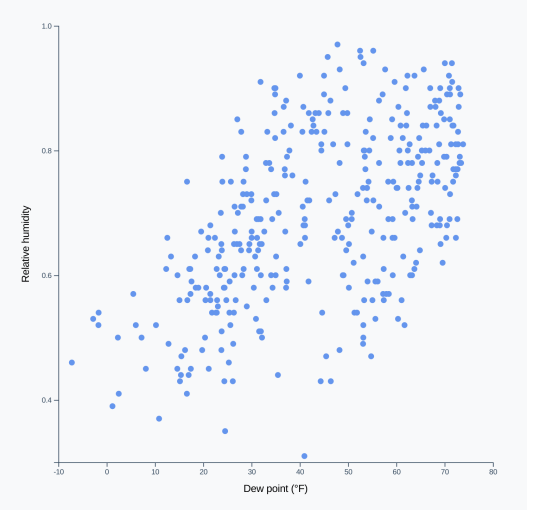
Look at that! You just made an interactive chart that gives users more information when they need it. Positioning tooltips is not a simple feat, so give yourself a pat on the back! Next up, we'll learn an even fancier method for making it easy for users to get tooltips even for small, close-together elements.

Scatter plot

Let's level up and add tooltips to a scatter plot. Navigate to

`/code/05-interactions/3-scatter/draft/` in your browser and open up the `/code/05-interactions/3-scatter/draft/scatter.js` file.

You should see the scatter plot we made in **Chapter 2** — no interactions here yet.



Scatter plot

We want a tooltip to give us more information when we hover over a point in our chart. Let's go through the steps from last chapter.

At the bottom of the file, we'll select all of our **<circle/>** elements and add a `mousenter` and a `mouseleave` event.

```
bounds.selectAll("circle")
    .on("mouseenter", onMouseEnter)
    .on("mouseleave", onMouseLeave)
```

We know that we'll need to modify our `#tooltip` element, so let's assign that to a variable. Let's also define our `onMouseEnter()` and `onMouseLeave()` functions.

```
const tooltip = d3.select("#tooltip")
function onMouseEnter(datum, index) {
}

function onMouseLeave() {
}
```

Let's first fill out our `onMouseEnter()` function. We want to display two values:

- the metric on our x axis (**dew point**), and
- the metric on our y axis (**humidity**).

For both metrics, we'll want to define a string formatter using `d3.format()`. Then we'll use that formatter to set the **text** value of the relevant **** in our tooltip.

```
function onMouseEnter(datum) {
  const formatHumidity = d3.format(".2f")
  tooltip.select("#humidity")
      .text(formatHumidity(yAccessor(datum)))

  const formatDewPoint = d3.format(".2f")
  tooltip.select("#dew-point")
      .text(formatDewPoint(xAccessor(datum)))

}
```

Let's add an extra bit of information at the bottom of this function — users will probably want to know the date of the hovered point. Our data point's date is formatted as a string, but not in a very human-readable format (for example, "2019-01-01"). Let's use `d3.timeParse` to turn that string into a date that we can re-format.

```
const dateParser = d3.timeParse("%Y-%m-%d")
console.log(dateParser(datum.date))
```

Now we need to turn our date object into a friendlier string. The **d3-time-format**[30] module can help us out here! `d3.timeFormat()` will take a date formatter string and return a formatter function.

The date formatter string uses the same syntax as `d3.timeParse` — it follows four rules:

1. it will return the string verbatim, other than specific directives,
2. these directives contain a percent sign and a letter,

[30] https://github.com/d3/d3-time-format

3. usually the letter in a directive has two formats: lowerbase (abbreviated) and uppercase (full), and
4. a dash (-) between the percent sign and the letter prevents padding of numbers.

For example, `d3.timeFormat("%Y")(new Date())` will return the current year.

Let's learn a few handy directives:

- `%Y`: the full year
- `%y`: the last two digits of the year
- `%m`: the padded month (eg. "01")
- `%-m`: the non-padded month (eg. "1")
- `%B`: the full month name
- `%b`: the abbreviated month name
- `%A`: the full weekday name
- `%a`: the abbreviated weekday name
- `%d`: the day of the month

See the full list of directives at https://github.com/d3/d3-time-format[31].

Now, let's create a formatter string that prints out a friendly date.

```
const dateParser = d3.timeParse("%Y-%m-%d")
const formatDate = d3.timeFormat("%B %A %-d, %Y")
console.log(formatDate(dateParser(datum.date)))
```

Much better! Let's plug that in to our tooltip.

```
const dateParser = d3.timeParse("%Y-%m-%d")
const formatDate = d3.timeFormat("%B %A %-d, %Y")
tooltip.select("#date")
    .text(formatDate(dateParser(datum.date)))
```

Next, we'll grab the `x` and `y` value of our dot , offset by the **top** and **left** margins.

[31] https://github.com/d3/d3-time-format

```
const x = xScale(xAccessor(datum))
  + dimensions.margin.left
const y = yScale(yAccessor(datum))
  + dimensions.margin.top
```

Just like with our bars, we'll use `calc()` to add these values to the percentage offsets needed to shift the tooltip. Remember, this is necessary so that we're positioning its arrow, not the top left corner.

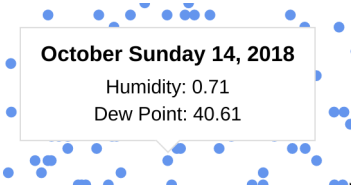
```
tooltip.style("transform", `translate(`
  + `calc( -50% + ${x}px),`
  + `calc(-100% + ${y}px)`
  + `)`)
```

Lastly, we'll make our tooltip visible and hide it when we mouse out of our dot.

```
  tooltip.style("opacity", 1)
}

function onMouseLeave() {
  tooltip.style("opacity", 0)
}
```

Nice! Adding a tooltip was much faster the second time around, wasn't it?



scatter plot tooltip

Those tiny dots are hard to hover over, though. The small hover target makes us focus really hard to move our mouse *exactly* over a point. To make things worse, our tooltip disappears when moving between points, making the whole interaction a little jerky.

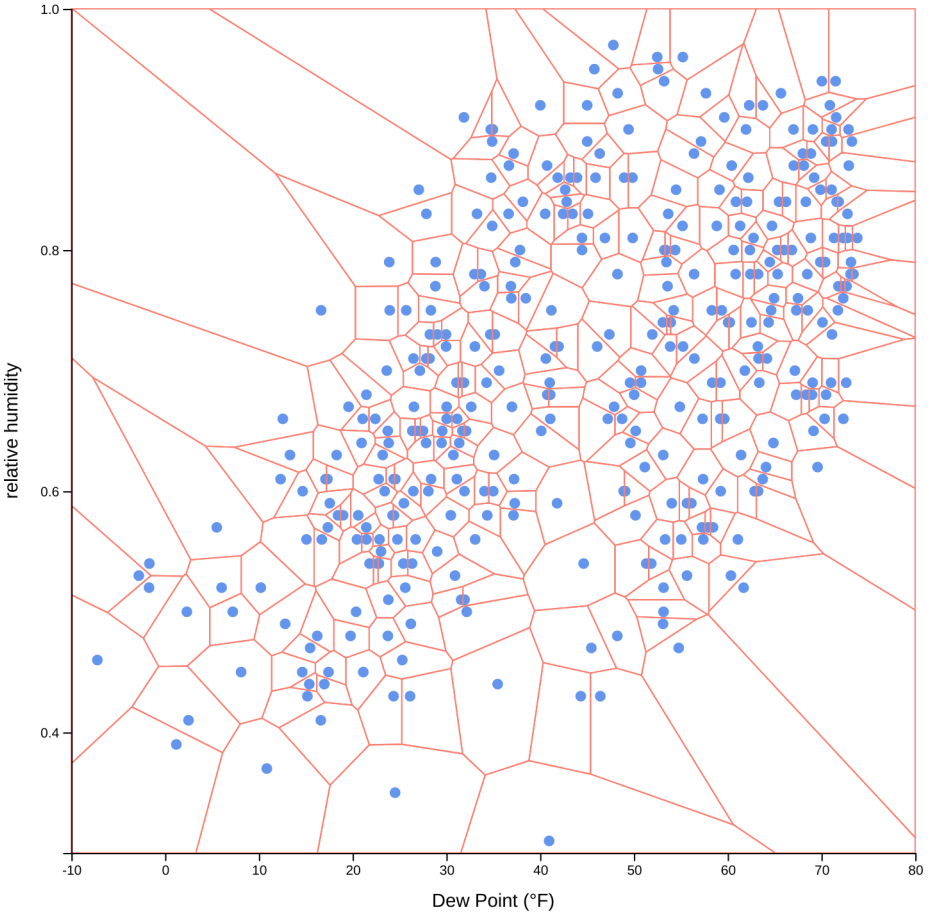
Don't worry! We have a very clever solution to this problem.

Voronoi

Let's talk briefly about **voronoi diagrams**. For every location on our scatter plot, there is a dot that is the closest. A voronoi diagram partitions a plane into regions based on the closest point. Any location within each of these parts agrees on the closest point.

Voronoi are useful in many fields — from creating art to detecting neuromuscular diseases to developing predictive models for forest fires.

Let's look at what our scatter plot would look like when split up with a voronoi diagram.



scatter plot with voronoi

See how each point in our scatter plot is inside of a cell? If you chose any location in that cell, that point would be the closest.

There is a voronoi generator built into the main d3 bundle: `d3-voronoi`[32]. However, this module is deprecated and the speedier `d3-delaunay`[33] is recommended instead.

We've included the minified version of this module at `/3-scatter/d3-delaunay.min.js` and loaded the script in our `index.html` file, so we'll already have access to

[32] https://github.com/d3/d3-voronoi
[33] https://github.com/d3/d3-delaunay

`d3.Delaunay`.

Enough talking, let's create our own diagram! Let's add some code at the end of the `Draw data` step, right before the `Draw peripherals` step. Because this is an external library, the API is a bit different from other d3 code. Instead of creating a voronoi generator, we'll create a new **Delaunay triangulation**. A delaunay triangulation[34] is a way to join a set of points to create a triangular mesh. To create this, we can pass `d3.Delaunay.from()`[35] three parameters:

1. our dataset,
2. an x accessor function, and
3. a y accessor function.

```
const delaunay = d3.Delaunay.from(
  dataset,
  d => xScale(xAccessor(d)),
  d => yScale(yAccessor(d)),
)
```

Now we want to turn our **delaunay triangulation** into a voronoi diagram – thankfully our triangulation has a `.voronoi()` method.

```
const voronoi = delaunay.voronoi()
```

Let's bind our data and add a `<path>` for each of our data points with a class of "voronoi" (for styling with our `styles.css` file).

```
bounds.selectAll(".voronoi")
  .data(dataset)
  .enter().append("path")
    .attr("class", "voronoi")
```

We can create each path's `d` attribute string by passing `voronoi.renderCell()` the *index of our data point.*

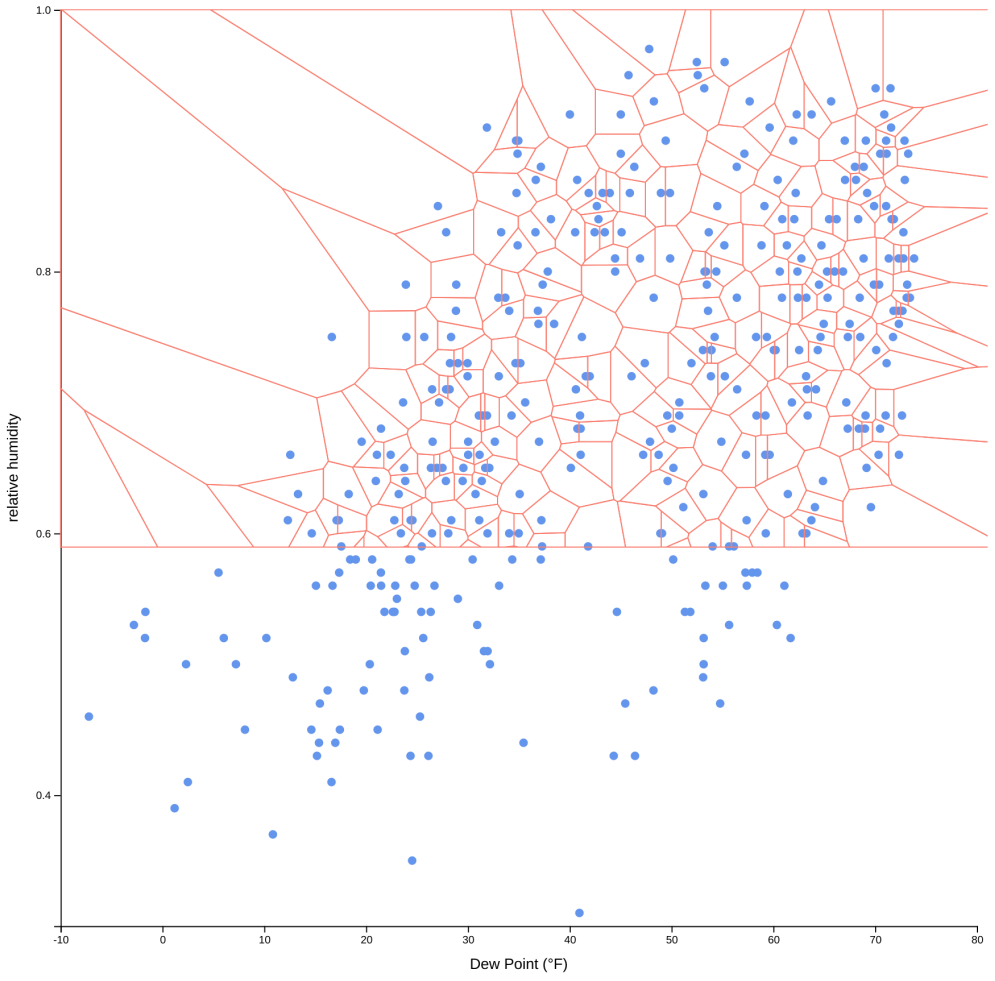
[34] https://en.wikipedia.org/wiki/Delaunay_triangulation
[35] https://github.com/d3/d3-delaunay#delaunay_from

```
bounds.selectAll(".voronoi")
    // ...
    .attr("d", (d,i) => voronoi.renderCell(i))
```

Lastly, let's give our paths a `stroke` value of `salmon` so that we can look at them.

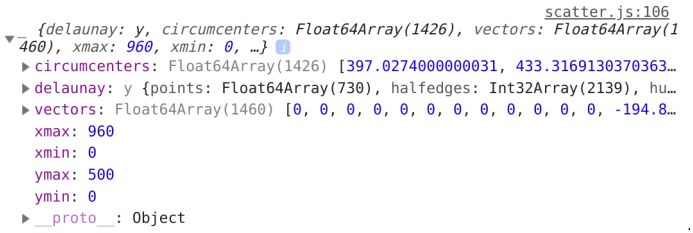
```
bounds.selectAll(".voronoi")
    // ...
    .attr("stroke", "salmon")
```

Now when we refresh our webpage, our scatter plot will be split into voronoi cells!



voronoi paths

Hmm, our voronoi diagram is wider and shorter than our chart. This is because it has no concept of the size of our bounds, and is using the default size of `960` pixels wide and `500` pixels tall, which we can see if we log out our `voronoi` object.

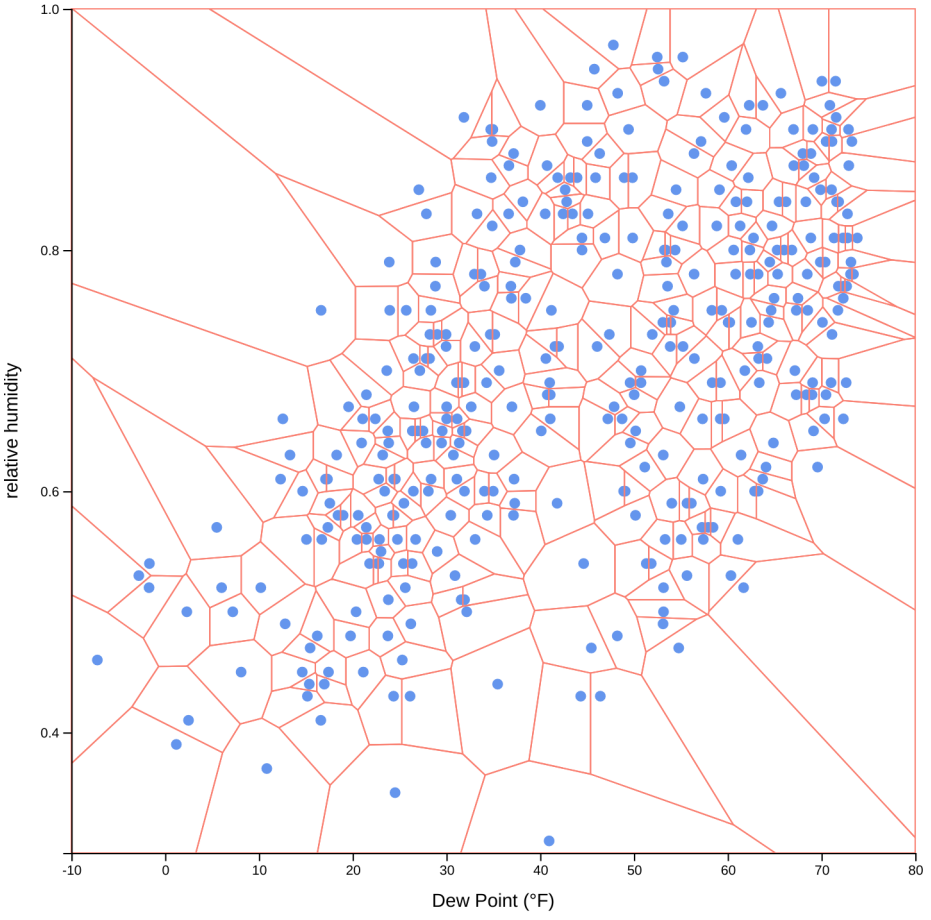


voronoi paths

Let's specify the size of our diagram by setting our `voronoi`'s `.xmax` and `.ymax` values (before we draw our `<path>`s).

```
const voronoi = delaunay.voronoi()
voronoi.xmax = dimensions.boundedWidth
voronoi.ymax = dimensions.boundedHeight
```

Voila! Now our diagram is the correct size.



scatter plot with voronoi

What we want is to capture hover events for our paths instead of an individual dot. This will be much easier to interact with because of the contiguous, large hover targets.

Let's remove that last line where we set the `stroke` (`.attr("stroke", "salmon")`) so our voronoi cells are invisible. Next, we'll update our interactions, starting by moving our `mouseenter` and `mouseleave` events from the dots to our voronoi paths.

> Note that the mouse events on our dots won't be triggered anymore, since they're covered by our voronoi paths.

```
bounds.selectAll(".voronoi")
    // ...
    .on("mouseenter", onMouseEnter)
    .on("mouseleave", onMouseLeave)
```

When we refresh our webpage, notice how much easier it is to target a specific dot!

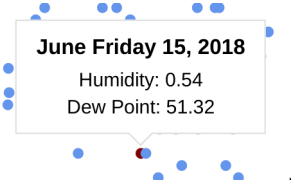
Changing the hovered dot's color

Now that we don't need to directly hover over a dot, it can be a bit unclear which dot we're getting data about. Let's make our dot change color and grow on hover.

The naive approach would involve selecting the corresponding **circle** and changing its fill. Note that d3 selection objects have a `.filter()` method that mimics a native Array's.

```
function onMouseEnter(datum) {
  bounds.selectAll("circle")
    .filter(d => d == datum)
      .style("fill", "maroon")
```

However, we'll run into an issue here. Remember that SVG elements' z-index is determined by their position in the DOM. We can't change our dots' order easily on hover, so any dot drawn after our hovered dot will obscure it.



scatter plot overlap

Instead, we'll draw a completely new dot which will appear on top.

```
function onMouseEnter(datum) {
  const dayDot = bounds.append("circle")
      .attr("class", "tooltipDot")
      .attr("cx", xScale(xAccessor(datum)))
      .attr("cy", yScale(yAccessor(datum)))
      .attr("r", 7)
      .style("fill", "maroon")
      .style("pointer-events", "none")
```

Let's remember to remove this new dot on mouse leave.

```
function onMouseLeave() {
  d3.selectAll(".tooltipDot")
    .remove()
```

Now when we trigger a tooltip, we can see our hovered dot clearly!